

TOPIC 3: DIVIDE AND CONQUER

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

```
[1]
✓ Os
def find_min_max(arr):
    minimum = arr[0]
    maximum = arr[0]

    for x in arr:
        if x < minimum:
            minimum = x
        if x > maximum:
            maximum = x

    return minimum, maximum

arr = [5, 7, 3, 4, 9, 12, 6, 2]

minimum, maximum = find_min_max(arr)

print("Min =", minimum)
print("Max =", maximum)

... Min = 2
   Max = 12
```

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

```
[3]
✓ Os
def find_min_max_sorted(arr):
    return arr[0], arr[-1]

arr = [2, 4, 6, 8, 10, 12, 14, 18]

minimum, maximum = find_min_max_sorted(arr)

print("Min =", minimum)
print("Max =", maximum)

... Min = 2
   Max = 18
```

3. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

```

[4]
✓ Os
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

arr = [31,23,35,27,11,21,15,28]

print(merge_sort(arr))

... [11, 15, 21, 23, 27, 28, 31, 35]

```

4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

```

right = merge_sort(arr[mid:])

result = []
i = 0
j = 0

while i < len(left) and j < len(right):
    count += 1

    if left[i] <= right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

result.extend(left[i:])
result.extend(right[j:])

return result

arr = [12,4,78,23,45,67,89,1]

count = 0
result = merge_sort(arr)

print("Sorted Array:", result)
print("Number of Comparisons:", count)

... Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Number of Comparisons: 16

```

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

```

[6]
✓ Os
▶ Run all

if i < j:
    arr[i], arr[j] = arr[j], arr[i]
else:
    break

arr[low], arr[j] = arr[j], arr[low]

return j

def quick_sort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)

        print("After partition:", arr)

        quick_sort(arr, low, pivot_index - 1)
        quick_sort(arr, pivot_index + 1, high)

arr = [10,16,8,12,15,6,3,9,5]

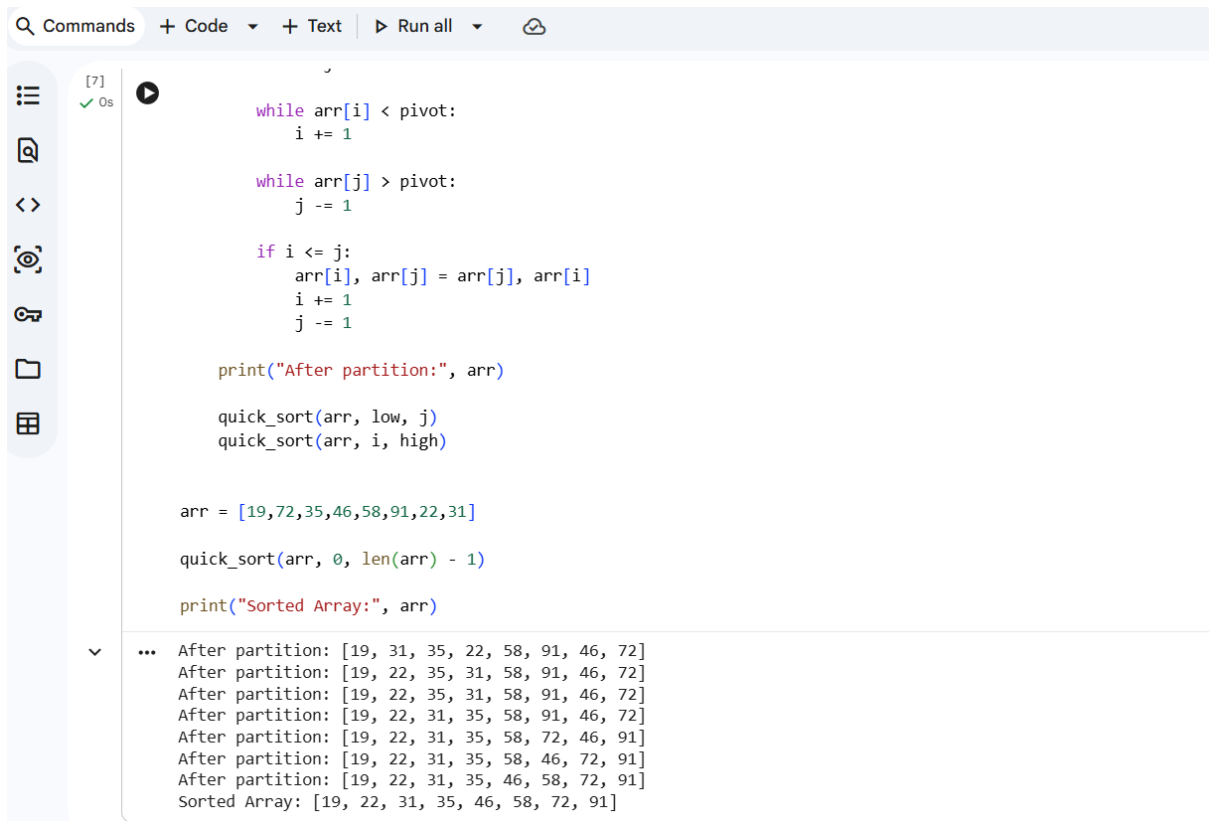
quick_sort(arr, 0, len(arr) - 1)

print("Sorted Array:", arr)

... After partition: [6, 5, 8, 9, 3, 10, 15, 12, 16]
After partition: [3, 5, 6, 9, 8, 10, 15, 12, 16]
After partition: [3, 5, 6, 9, 8, 10, 15, 12, 16]
After partition: [3, 5, 6, 8, 9, 10, 15, 12, 16]
After partition: [3, 5, 6, 8, 9, 10, 12, 15, 16]
Sorted Array: [3, 5, 6, 8, 9, 10, 12, 15, 16]

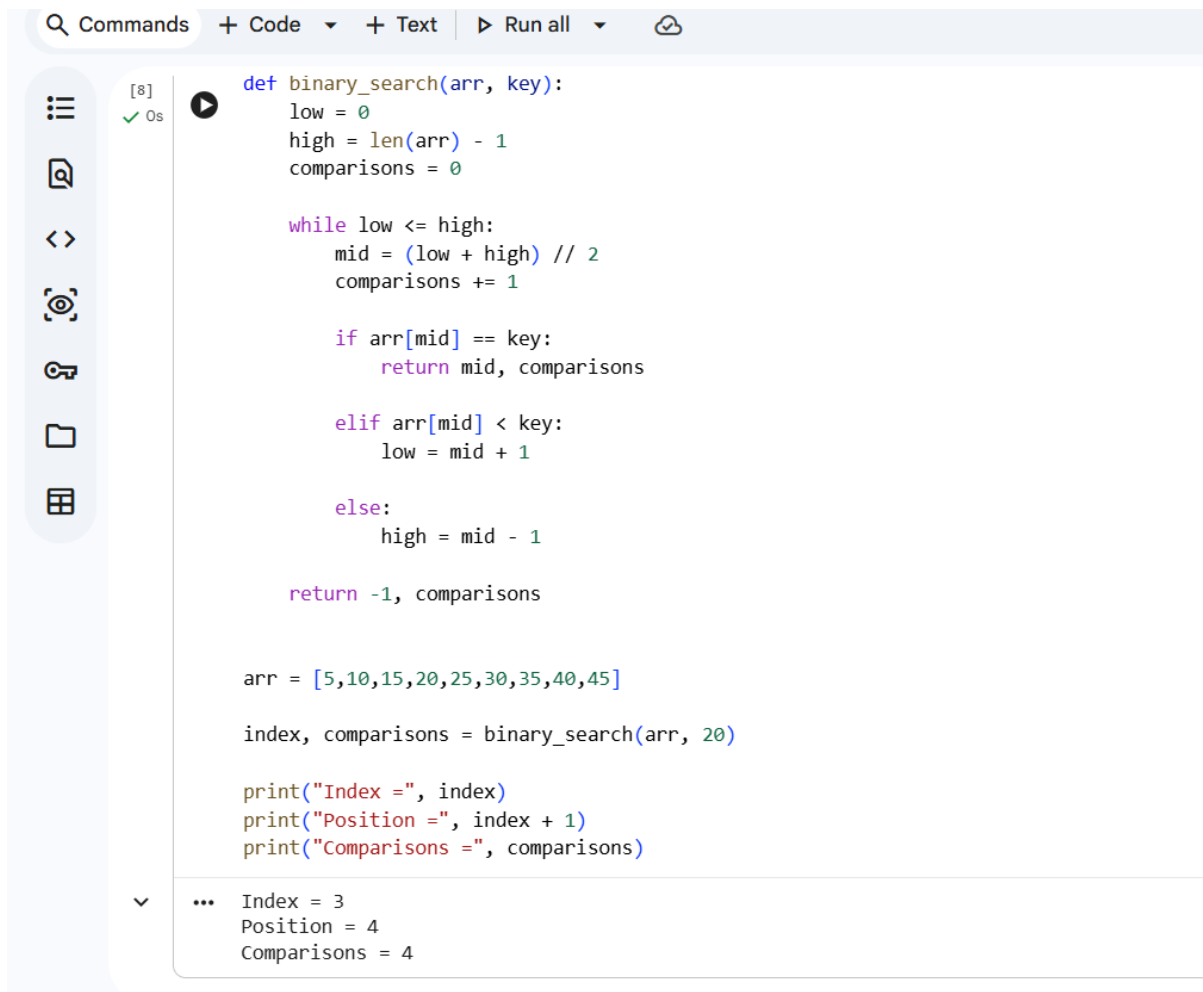
```

6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.



```
Q Commands + Code + Text ▶ Run all ☁  
[7] ✓ Os  
while arr[i] < pivot:  
    i += 1  
  
while arr[j] > pivot:  
    j -= 1  
  
if i <= j:  
    arr[i], arr[j] = arr[j], arr[i]  
    i += 1  
    j -= 1  
  
print("After partition:", arr)  
  
quick_sort(arr, low, j)  
quick_sort(arr, i, high)  
  
arr = [19,72,35,46,58,91,22,31]  
  
quick_sort(arr, 0, len(arr) - 1)  
  
print("Sorted Array:", arr)  
  
... After partition: [19, 31, 35, 22, 58, 91, 46, 72]  
After partition: [19, 22, 35, 31, 58, 91, 46, 72]  
After partition: [19, 22, 35, 31, 58, 91, 46, 72]  
After partition: [19, 22, 31, 35, 58, 91, 46, 72]  
After partition: [19, 22, 31, 35, 58, 72, 46, 91]  
After partition: [19, 22, 31, 35, 58, 46, 72, 91]  
After partition: [19, 22, 31, 35, 46, 58, 72, 91]  
Sorted Array: [19, 22, 31, 35, 46, 58, 72, 91]
```

7. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.



The screenshot shows a code editor interface with a top bar containing 'Commands', '+ Code', '+ Text', 'Run all', and a cloud icon. On the left is a sidebar with icons for file explorer, search, and other tools. The main editor area displays a Python function `binary_search(arr, key)` that implements a binary search algorithm. The function initializes `low = 0` and `high = len(arr) - 1`, and counts comparisons. It uses a `while` loop to find the `key` in the sorted array `arr`. Below the function, an array `arr = [5, 10, 15, 20, 25, 30, 35, 40, 45]` is defined, and the function is called with `binary_search(arr, 20)`. The output shows the index (3), position (4), and number of comparisons (4).

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if arr[mid] == key:
            return mid, comparisons

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1, comparisons

arr = [5, 10, 15, 20, 25, 30, 35, 40, 45]

index, comparisons = binary_search(arr, 20)

print("Index =", index)
print("Position =", index + 1)
print("Comparisons =", comparisons)
```

... Index = 3
Position = 4
Comparisons = 4

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?



```
[10]
✓ Os
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        print("Low =", low, "High =", high, "Mid =", mid)

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1

arr = [3,9,14,19,25,31,42,47,53]

index = binary_search(arr, 31)

print("Position =", index + 1)

... Low = 0 High = 8 Mid = 4
    Low = 5 High = 8 Mid = 6
    Low = 5 High = 5 Mid = 5
    Position = 6
```

9. Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

[11]
✓ Os

```
def k_closest(points, k):  
    points.sort(key=lambda p: p[0]**2 + p[1]**2)  
  
    return points[:k]  
  
print(k_closest([[1,3],[-2,2],[5,8],[0,1]], 2))  
  
print(k_closest([[1,3],[-2,2]], 1))  
  
print(k_closest([[3,3],[5,-1],[-2,4]], 2))
```

✓

```
[[0, 1], [-2, 2]]  
[[-2, 2]]  
[[3, 3], [-2, 4]]
```

10. Given four lists A, B, C, D of integer values, Write a program to compute how many tuples $n(i, j, k, l)$ there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.

<>

[12]
✓ Os

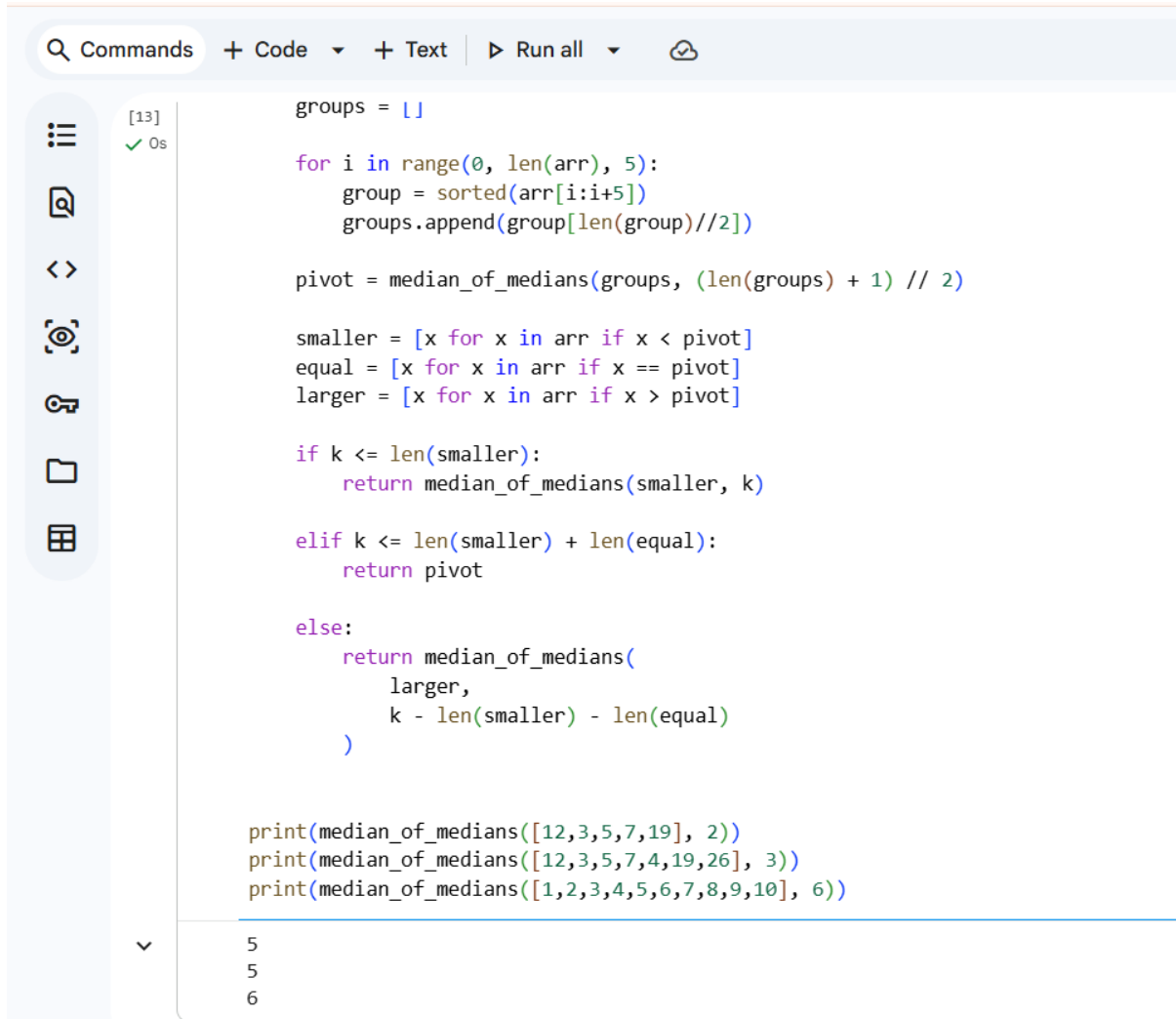
```
def four_sum_count(A, B, C, D):  
    count = 0  
  
    for a in A:  
        for b in B:  
            for c in C:  
                for d in D:  
                    if a + b + c + d == 0:  
                        count += 1  
  
    return count
```

```
A = [1,2]  
B = [-2,-1]  
C = [-1,2]  
D = [0,2]  
  
print(four_sum_count(A,B,C,D))  
  
print(four_sum_count([0],[0],[0],[0]))
```

✓

```
2  
1
```

11. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.



```
groups = []

for i in range(0, len(arr), 5):
    group = sorted(arr[i:i+5])
    groups.append(group[len(group)//2])

pivot = median_of_medians(groups, (len(groups) + 1) // 2)

smaller = [x for x in arr if x < pivot]
equal = [x for x in arr if x == pivot]
larger = [x for x in arr if x > pivot]

if k <= len(smaller):
    return median_of_medians(smaller, k)

elif k <= len(smaller) + len(equal):
    return pivot

else:
    return median_of_medians(
        larger,
        k - len(smaller) - len(equal)
    )

print(median_of_medians([12,3,5,7,19], 2))
print(median_of_medians([12,3,5,7,4,19,26], 3))
print(median_of_medians([1,2,3,4,5,6,7,8,9,10], 6))
```

5
5
6

12. To Implement a function `median_of_medians(arr, k)` that takes an unsorted array `arr` and an integer `k`, and returns the k-th smallest element in the array.


```
[14] ✓ 0s
pivot = median_of_medians(
    medians,
    (len(medians) + 1) // 2
)

smaller = [x for x in arr if x < pivot]
equal = [x for x in arr if x == pivot]
larger = [x for x in arr if x > pivot]

if k <= len(smaller):
    return median_of_medians(smaller, k)

elif k <= len(smaller) + len(equal):
    return pivot

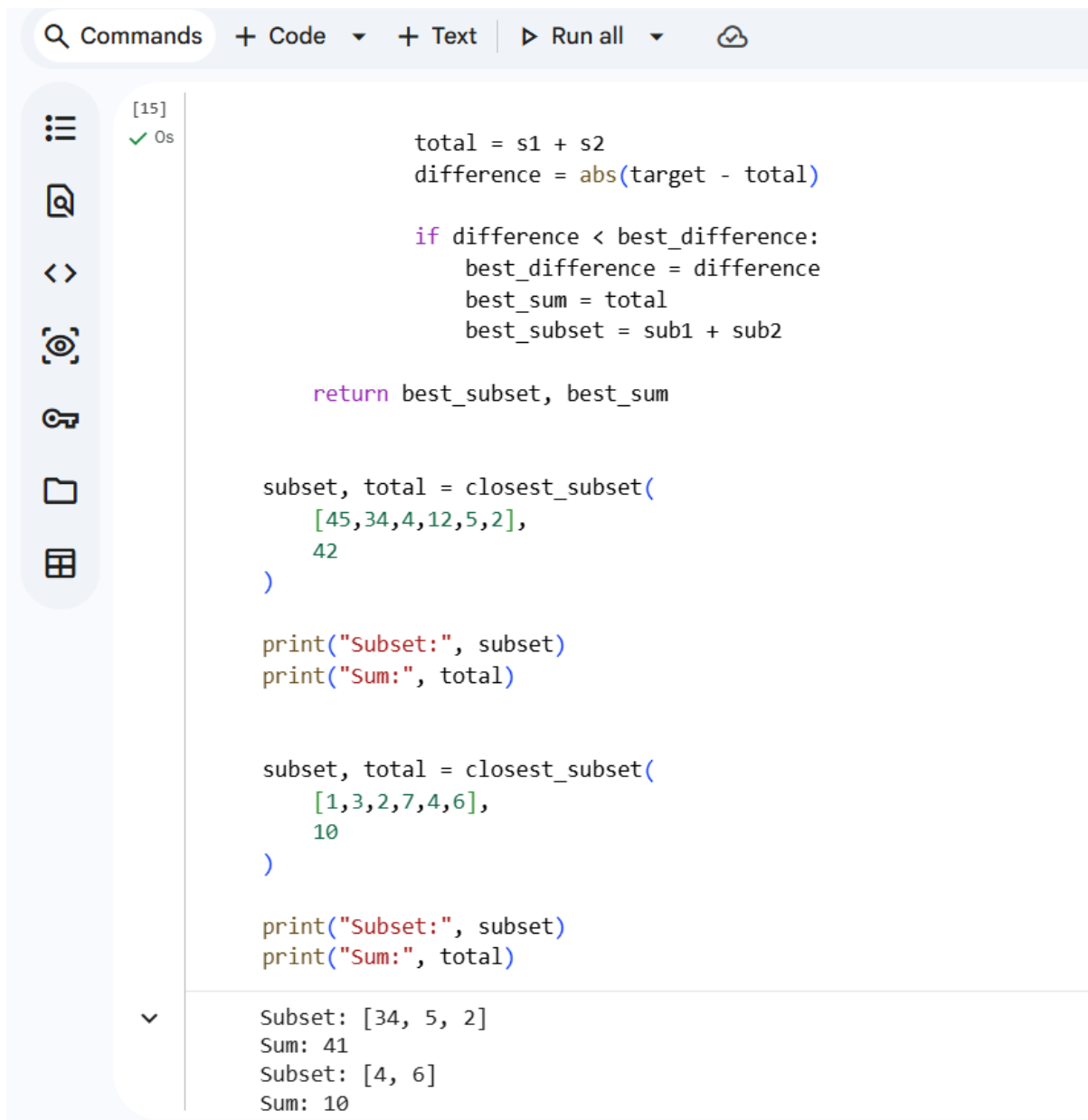
else:
    return median_of_medians(
        larger,
        k - len(smaller) - len(equal)
    )

print(median_of_medians(
    [1,2,3,4,5,6,7,8,9,10], 6
))

print(median_of_medians(
    [23,17,31,44,55,21,20,18,19,27], 5
))
```

6
21

13. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet in the Middle technique to efficiently find this subset.



```
[15]
✓ 0s

        total = s1 + s2
        difference = abs(target - total)

        if difference < best_difference:
            best_difference = difference
            best_sum = total
            best_subset = sub1 + sub2

        return best_subset, best_sum

subset, total = closest_subset(
    [45,34,4,12,5,2],
    42
)

print("Subset:", subset)
print("Sum:", total)

subset, total = closest_subset(
    [1,3,2,7,4,6],
    10
)

print("Subset:", subset)
print("Sum:", total)

▼
Subset: [34, 5, 2]
Sum: 41
Subset: [4, 6]
Sum: 10
```

14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E , determine if there is any subset that sums exactly to E . Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E , otherwise return false.

```
[16]
✓ 0s

def get_sums(arr):
    sums = set()

    for r in range(len(arr) + 1):
        for combo in combinations(arr, r):
            sums.add(sum(combo))

    return sums

def subset_sum_exists(arr, target):
    mid = len(arr) // 2

    left = arr[:mid]
    right = arr[mid:]

    left_sums = get_sums(left)
    right_sums = get_sums(right)

    for s in left_sums:
        if target - s in right_sums:
            return True

    return False

print(subset_sum_exists([1,3,9,2,7,12], 15))

print(subset_sum_exists([3,34,4,12,5,2], 15))

True
True
```

15 Given two 2×2 Matrices A and B

$A = \begin{pmatrix} 1 & 7 \\ 3 & 5 \end{pmatrix}$ $B = \begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix}$

Use Strassen's matrix multiplication algorithm to compute the product matrix C such that

$C = A \times B$.

```
Q Commands + Code + Text ▶ Run all ☁

[17] ✓ 0s ▶
M4 = a ~ (g - e)
M5 = (a + b) * h
M6 = (c - a) * (e + f)
M7 = (b - d) * (g + h)

C11 = M1 + M4 - M5 + M7
C12 = M3 + M5
C21 = M2 + M4
C22 = M1 - M2 + M3 + M6

return [
    [C11, C12],
    [C21, C22]
]

A = [
    [1, 7],
    [3, 5]
]

B = [
    [6, 8],
    [4, 2]
]

C = strassen(A, B)

for row in C:
    print(row)

... [34, 22]
    [38, 34]
```

16 Given two integers $X=1234$ and $Y=5678$: Use the Karatsuba algorithm to compute the product $Z=X \times Y$.



[18]
✓ Os

```
if x < 10 or y < 10:
    return x * y

n = max(len(str(x)), len(str(y)))
m = n // 2

high_x = x // (10 ** m)
low_x = x % (10 ** m)

high_y = y // (10 ** m)
low_y = y % (10 ** m)

z0 = karatsuba(low_x, low_y)
z1 = karatsuba(low_x + high_x, low_y + high_y)
z2 = karatsuba(high_x, high_y)

return (
    z2 * (10 ** (2 * m))
    + (z1 - z2 - z0) * (10 ** m)
    + z0
)

x = 1234
y = 5678

result = karatsuba(x, y)

print("Product =", result)
```



Product = 7006652